

Relational Algebra

Chapter # 07

Relational Algebra & Calculus

- Relational algebra and relational calculus are formal languages associated with the relational model.
- Informally, relational algebra is a (high-level) procedural language and relational calculus a non-procedural language.
 - Relational Algebra: More operational (procedural), very useful for representing execution plans.
 - Relational Calculus: Lets users describe what they want, rather than how to compute it: Non-operational, declarative.
- However, formally both are equivalent to one another.

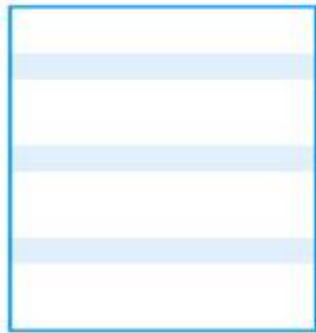
Relational Algebra

- The basic set of operations for the relational model is the relational algebra.
 - **enable the specification of basic retrievals**
- Relational algebra operations work on one or more relations to define another relation without changing the original relations.
- Both operands and results are relations, so output from one operation can become input to another operation.
- Allows expressions to be nested, just as in arithmetic.

Relational Algebra

- Five basic operations in relational algebra: Selection, Projection, Cartesian product, Union, and Set Difference.
- These perform most of the data retrieval operations needed.
- Also have Join, Intersection, and Division operations, which can be expressed in terms of 5 basic operations.

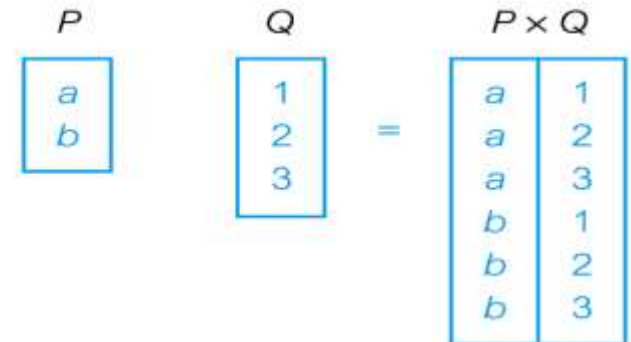
Relational Algebra Operations



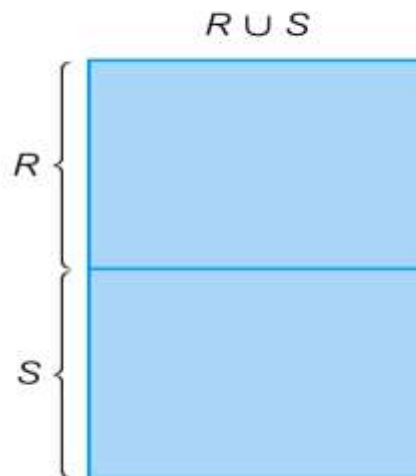
(a) Selection



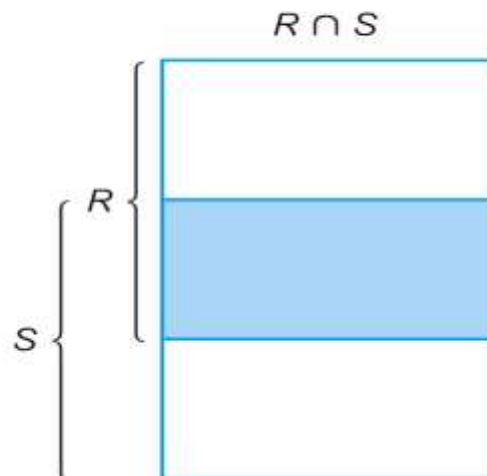
(b) Projection



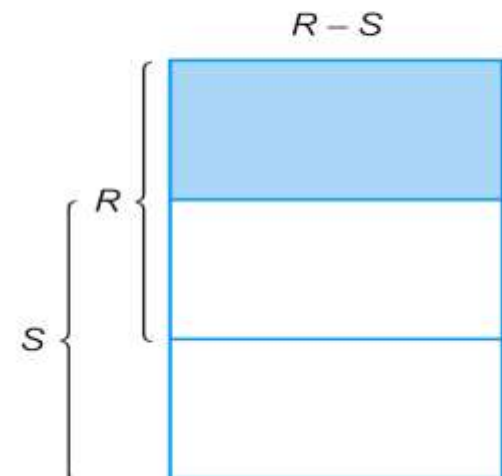
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

Unary Relational Operations

SELECT Operation: used to select a *subset* of the tuples from a relation that satisfy a **selection condition**. It is a filter that keeps only those tuples that satisfy a qualifying condition.

Examples:

$\sigma_{\text{DNO} = 4}(\text{EMPLOYEE})$

$\sigma_{\text{SALARY} > 30,000}(\text{EMPLOYEE})$

- denoted by $\sigma_{\text{<selection condition>}}(R)$ where the symbol σ (sigma) is used to denote the select operator, and the selection condition is a *Boolean expression* specified on the attributes of relation R

SELECT Operation Properties

The SELECT operation $\sigma_{\langle \text{selection condition} \rangle}(R)$ produces a relation S that has the same schema as R

The SELECT operation σ is **commutative**; i.e.,

$$\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(R)) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))$$

A cascaded SELECT operation **may be applied in any order**; i.e.,

$$\begin{aligned} &\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition3} \rangle}(R))) \\ &= \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition3} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))) \end{aligned}$$

A cascaded SELECT operation may be replaced by a single selection with a conjunction of all the conditions; i.e.,

$$\begin{aligned} &\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition3} \rangle}(R))) \\ &= \sigma_{\langle \text{condition1} \rangle \text{ AND } \langle \text{condition2} \rangle \text{ AND } \langle \text{condition3} \rangle}(R) \end{aligned}$$

Selection Condition

Operators: $<$, $\leq$, $\geq$, $>$, $=$, $\neq$

Simple selection condition:

- *$\langle \text{attribute} \rangle$ operator $\langle \text{constant} \rangle$*
- *$\langle \text{attribute} \rangle$ operator $\langle \text{attribute} \rangle$*
- *$\langle \text{condition} \rangle$ AND $\langle \text{condition} \rangle$*
- *$\langle \text{condition} \rangle$ OR $\langle \text{condition} \rangle$*
- *NOT $\langle \text{condition} \rangle$*

Selection Condition

$\sigma_{Id > 3000 \text{ OR } Hobby = 'hiking'}(Person)$

$\sigma_{Id > 3000 \text{ AND } Id < 3999}(Person)$

$\sigma_{NOT(Hobby = 'hiking')}(Person)$

$\sigma_{Hobby \neq 'hiking'}(Person)$

<i>Id</i>	<i>Name</i>	<i>Address</i>	<i>Hobby</i>
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Unary Relational Operations

PROJECT Operation: selects certain *columns* from the table and discards the others.

Example:

$\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE})$

The general form of the project operation is:

$\pi_{\langle \text{attribute list} \rangle}(\text{R})$

where π is the symbol used to represent the project operation and $\langle \text{attribute list} \rangle$ is the desired list of attributes.

PROJECT *removes duplicate tuples*, so the result is a set of tuples and hence a valid relation.

PROJECT Operation Properties

- The number of tuples in the result of $\pi_{\langle \text{list} \rangle}(R)$ is always less or equal to the number of tuples in R .
- If attribute list includes a key of R , then the number of tuples is equal to the number of tuples in R .

SELECT and PROJECT Operations

(a) $\sigma_{(DNO=4 \text{ AND } SALARY>25000) \text{ OR } (DNO=5 \text{ AND } SALARY>30000)}(EMPLOYEE)$

(b) $\pi_{LNAME, FNAME, SALARY}(EMPLOYEE)$

RENAME operation

RENAME operation

Rename attributes in intermediate results

$$\rho_{S(B_1, B_2, \dots, B_n)}(R)$$
$$\rho_S(R)$$
$$\rho_{(B_1, B_2, \dots, B_n)}(R)$$

where the symbol $\rho(\text{rho})$ is used to denote the RENAME operator, S is the new relation name, and B1, B2, ..., Bn are the new attribute names.

RENAME operation

- The first expression renames both the relation and its attributes

$\rho_S(B_1, B_2, \dots, B_n)(R)$

- The second renames the relation only

$\rho_S(R)$

- The third renames the attributes only.

$\rho(B_1, B_2, \dots, B_n)(R)$

If the attributes of R are $(A_1, A_2, \dots, A_n)$ in that order, then each A_i is renamed as B_i .

Relational Algebra Operations from Set Theory

- The UNION, INTERSECTION, and MINUS Operations
- The CARTESIAN PRODUCT (or CROSS PRODUCT) Operation

Set Operators

- A relation is a *set* of tuples, so set operations apply:
 $\cap$, $\cup$, $-$ (set difference)
- Result of combining two relations with a set operator is a relation $\Rightarrow$ all elements are tuples with the same structure

UNION Operation

Denoted by $R \cup S$

Result is a relation that includes all tuples that are either in R or in S or in both. Duplicate tuples are eliminated.

Example: Retrieve the SSNs of all employees who either work in department 5 or directly supervise an employee who works in department 5:

$DEP5_EMPS \leftarrow \sigma_{DNO=5}(EMPLOYEE)$

$RESULT1 \leftarrow \pi_{SSN}(DEP5_EMPS)$

$RESULT2(SSN) \leftarrow \pi_{SUPERSSN}(DEP5_EMPS)$

$RESULT \leftarrow RESULT1 \cup RESULT2$

The union operation produces the tuples that are in either RESULT1 or RESULT2 or both. The two operands must be “type compatible”.

UNION Operation

Type (Union) Compatibility

The operand relations $R_1(A_1, A_2, \dots, A_n)$ and $R_2(B_1, B_2, \dots, B_n)$ must have the same number of attributes, and the domains of corresponding attributes must be compatible, i.e.

- $\text{dom}(A_i) = \text{dom}(B_i)$ for $i=1, 2, \dots, n$.

Intersection Operation

Denoted by $R \cap S$

The intersection operation also has the requirement that both the relations should be union compatible, which means they are of same degree and same domains.

- If R and S are two relations and we take intersection of these two relations then the resulting relation would be the set of tuples, which are in both R and S . Just like union intersection is also commutative.
- $R \cap S = S \cap R$

Example

Tables

Person (*SSN, Name, Address, Hobby*)

Professor (*Id, Name, Office, Phone*)

are not union compatible.

But

$\pi_{Name}(\text{Person})$ and $\pi_{Name}(\text{Professor})$

are union compatible

Set Difference

Set Difference (or MINUS) Operation

The result of this operation, denoted by $R - S$, is a relation that includes all tuples that are in R but not in S . The two operands must be "type compatible".

Relational Algebra Operations From Set Theory

- Union and intersection are *commutative operations*:

$$\mathbf{R \cup S = S \cup R, \text{ and } R \cap S = S \cap R}$$

- Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are *associative operations*; that is

$$\mathbf{R \cup (S \cup T) = (R \cup S) \cup T, \text{ and}}$$

$$\mathbf{(R \cap S) \cap T = R \cap (S \cap T)}$$

- The minus operation is *not commutative*; that is, in general

$$\mathbf{R - S \neq S - R}$$

Relational Algebra Operations from Set Theory

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

(b)

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

(c)

Fn	Ln
Susan	Yao
Ramesh	Shah

(d)

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang

(e)

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

Cartesian (Cross) Product

- If R and S are two relations, $R \times S$ is the set of all concatenated tuples $\langle x, y \rangle$, where x is a tuple in R and y is a tuple in S
 - R and S need not be union compatible
- $R \times S$ is expensive to compute

A	B	C	D	A	B	C	D
x1	x2	y1	y2	x1	x2	y1	y2
x3	x4	y3	y4	x1	x2	y3	y4
				x3	x4	y1	y2
				x3	x4	y3	y4
R		S		$R \times S$			

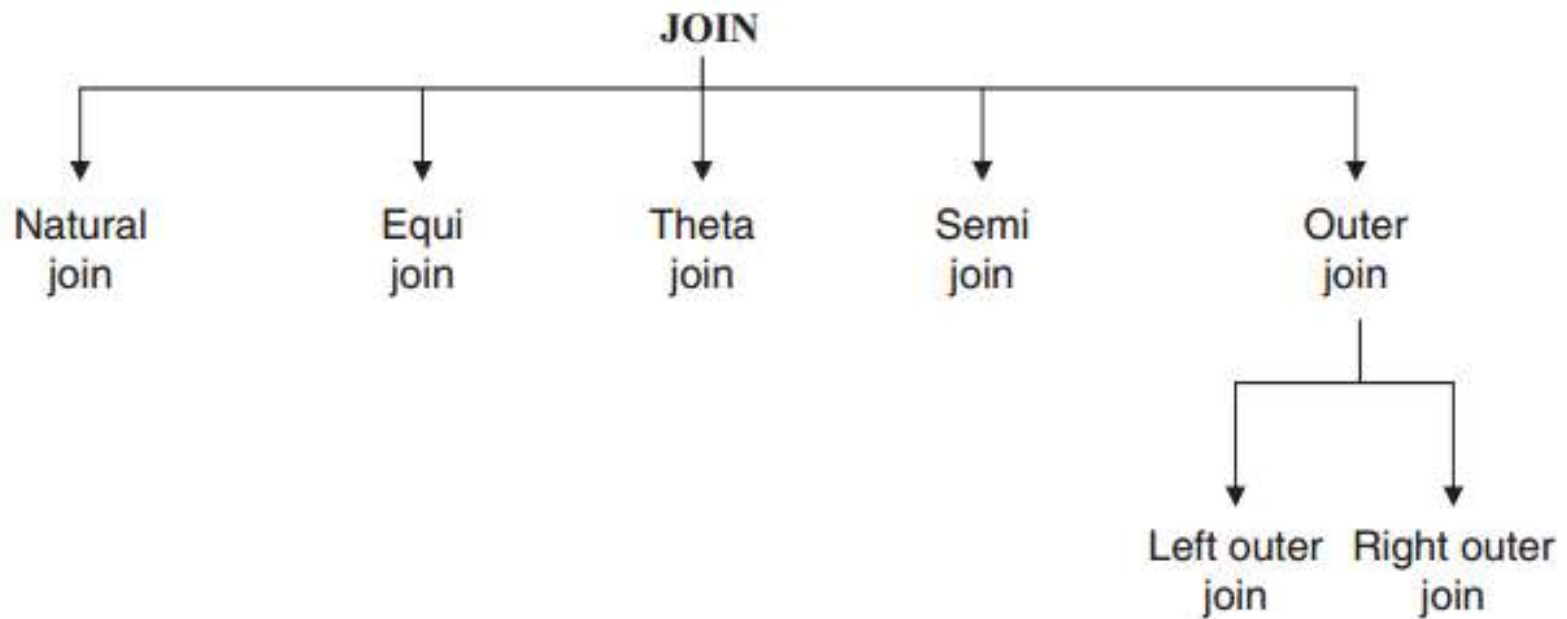
Binary Relational Operations: JOIN and DIVISION

- The JOIN Operation
- The EQUIJOIN and NATURAL JOIN variations of JOIN
- The DIVISION Operation

JOIN

- *Cartesian product* followed by *select* is commonly used to identify and select related tuples from two relations => called **JOIN**. It is denoted by a $\bowtie$
 - This operation is important for any relational database with more than a single relation, because it allows us to process *relationships* among relations.
 - The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is:
$$R \bowtie \langle \text{join condition} \rangle S$$
where R and S can be any relations that result from general *relational algebra expressions*

Types of JOIN



Natural Join

- The natural join performs an equi join of the two relations R and S over all common attribute
- a natural join will remove duplicate attribute
- In most systems a natural join will require that the attributes have the same name to identity the attributes to be used in the join.
- This may require a renaming mechanism.
- Denoted by *

Natural Join

- Input: Two relations (tables) R and S
- Notation: $R \bowtie S$
- Purpose: Relate rows from second table and
 - Enforce equality on all column attributes
 - Eliminate one copy of common attribute

NATURAL JOIN

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

NATURAL JOIN

In the following example, first we rename the Dnumber attribute

Of DEPARTMENT to Dnum—so that it has the same name as the Dnum attribute in PROJECT—and then we apply NATURAL JOIN:

$\text{PROJ_DEPT} \leftarrow \text{PROJECT} \bowtie \rho(\text{Dname}, \text{Dnum}, \text{Mgr_ssn}, \text{Mgr_start_date})(\text{DEPARTMENT})$

EQUIJOIN

- Only = comparison operator used
- Always have one or more pairs of attributes that have identical values in every tuple
- In the result of an EQUIJOIN we always have one or more pairs of attributes that have identical values in every tuple.

EQUIJOIN

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Figure 6.6

Result of the JOIN operation $\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{Mgr_ssn}=\text{Ssn}} \text{EMPLOYEE}$.

THETAJOIN

- A conditional join in which we impose condition other than equality condition.
- If equality condition is imposed then theta join become equi join. The symbol
- Θ stands for the comparison operator which could be $>$, $<$, $>=$, $<=$.
- Expression of Theta Join $\sigma_{\theta}(R \times S)$

OUTER JOINS

- Tuples without a matching(or related) tuple are eliminated from the JOIN result.
- Tuples with NULL values in the join attributes are also eliminated.
- This type of join, where tuples with no match are eliminated, is known as an inner join

Outer joins, can be used when we want to keep all the tuples in R, all those in S, or all those in both relations

OUTER JOINS

Types

- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- FULL OUTER JOIN

OUTER JOINS

1. Full outer join



2. Left outer join

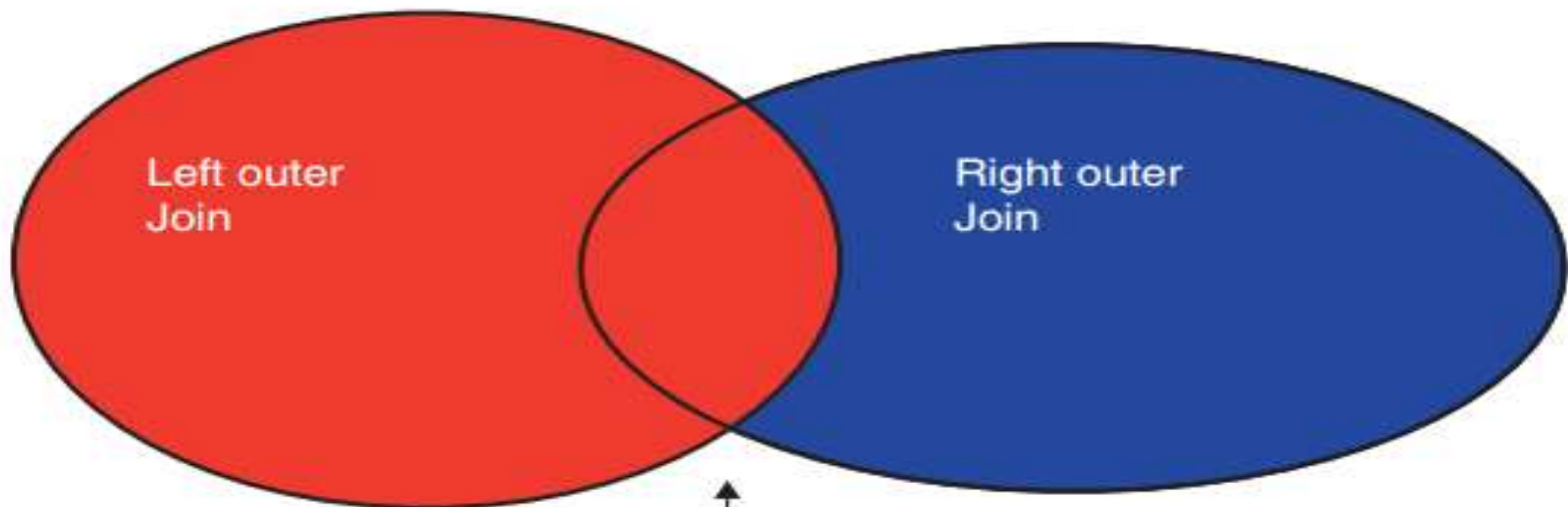


3. Right outer join



R

S



Unmatched rows
from left table

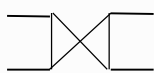
Matched rows

Unmatched rows from right
table

Left & Right outer join

- The left outer join operation keeps every tuple in the *first* or *left* relation R in $R \bowtie S$; if no matching tuple is found in S, then the attributes of S in the join result are “padded” with null values.
- A similar operation, right outer join, keeps every tuple in the *second* or *right* relation S in the result of $R \ltimes S$.

Full Outer Join

- A third operation, *full outer join*, denoted by  keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

Example

COURSE		
bkId	bkTitle	stId
B10001	Intro to Database Systems	S104
B10002	Programming Fundamentals	S101
B10003	Intro Data Structures	S101
B10004	Modern Operating Systems	S103
B10005	Computer Architecture	
B10006	Advanced Networks	S104

STUDENT	
stId	stName
S101	Ali Tahir
S103	Farah Hasan
S104	Farah Naz
S106	Asmat Dar
S107	Liaqat Ali

LEFT OUTER JOIN

COURSE left outer join STUDENT				
bkId	bkTitle	BOOK.stId	STUDENT.stId	stName
B10001	Intro to Database Systems	S104	S104	Farah Naz
B10002	Programming Fundamentals	S101	S101	Ali Tahir
B10003	Intro Data Structures	S101	S101	Ali Tahir
B10004	Modern Operating Systems	S103	S103	Farah Hasan
B10006	Advanced Networks	S104	S104	Farah Naz
B10005	Computer Architecture	Null	Null	Null

RIGHT OUTER JOIN

COURSE right outer join STUDENT				
bkId	bkTitile	BOOK.stId	STUDENT.stId	stName
B10001	Intro to Database Systems	S104	S104	Farah Naz
B10002	Programming Fundamentals	S101	S101	Ali Tahir
B10003	Intro Data Structures	S101	S101	Ali Tahir
B10004	Modern Operating Systems	S103	S103	Farah Hasan
B10006	Advanced Networks	S104	S104	Farah Naz
Null	Null	Null	S106	Asmat Dar
Null	Null	Null	S107	Liaqat Ali

FULL OUTER JOIN

COURSE outer join STUDENT				
bkId	bkTitle	BOOK.stId	STUDENT.stId	stName
B10001	Intro to Database Systems	S104	S104	Farah Naz
B10002	Programming Fundamentals	S101	S101	Ali Tahir
B10003	Intro Data Structures	S101	S101	Ali Tahir
B10004	Modern Operating Systems	S103	S103	Farah Hasan
B10006	Advanced Networks	S104	S104	Farah Naz
B10005	Computer Architecture	Null	Null	Null
Null	Null	Null	S106	Asmat Dar
Null	Null	Null	S107	Liaqat Ali

Semi Join

In semi join, first we take the natural join of two relations then we project the attributes of first table only. So after join and matching the common attribute of both relations only attributes of first relation are projected.

Example 2

FACULTY

facId	facName	dept	salary	rank
F234	Usman	CSE	21000	lecturer
F235	Tahir	CSE	23000	Asso Prof
F236	Ayesha	ENG	27000	Asso Prof
F237	Samad	ENG	32000	Professor

COURSE

crCode	crTitle	fld
C3456	Database Systems	F234
C3457	Financial Management	
C3458	Money & Capital Market	F236
C3459	Introduction to Accounting	F237

Semi Join

facId	facName	Dept	Salary	Rank
F234	Usman	CSE	21000	lecturer
F236	Ayesha	ENG	27000	Asso Prof
F237	Samad	ENG	32000	Professor

Division

- Suppose A has two groups of fields $\langle x, y \rangle$
- y fields are same fields in terms of domain as B
- $A/B = \langle x \rangle$ such as for every y value in a tuple of B there is $\langle x, y \rangle$ in A

Division

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

A

pno
p2

B1

sno
s1
s2
s3
s4

A/B1

pno
p2
p4

B2

sno
s1
s4

A/B2

pno
p1
p2
p4

B3

sno
s1

A/B3



Activate Windows

Go to PC settings to activate Windows

Division

- Identify all clients who have viewed all properties with three rooms.

$$(\Pi_{\text{clientNo, propertyNo}}(\text{Viewing})) \div (\Pi_{\text{propertyNo}}(\sigma_{\text{rooms} = 3}(\text{PropertyForRent})))$$

$\Pi_{\text{clientNo, propertyNo}}(\text{Viewing})$		$\Pi_{\text{propertyNo}}(\sigma_{\text{rooms}=3}(\text{PropertyForRent}))$	RESULT
clientNo	propertyNo	propertyNo	clientNo
CR56	PA14	PG4	CR56
CR76	PG4	PG36	
CR56	PG4		
CR62	PA14		
CR56	PG36		

Additional Relational Operations

Generalized Projection

The generalized projection operation extends the projection operation by allowing functions of attributes to be included in the projection list. The generalized form can be expressed as

$$\pi_{F_1, F_2, \dots, F_n}(R)$$

Generalized Projection

As an example, consider the relation

EMPLOYEE (Ssn, Salary, Deduction, Years_service)

A report may be required to show

Net Salary = Salary – Deduction,

Bonus = 2000 * Years_service, and

Tax = 0.25 * Salary.

Then a generalized projection combined with renaming may be used as follows:

REPORT $\leftarrow \rho_{(Ssn, Net_salary, Bonus, Tax)}(\pi_{Ssn, Salary - Deduction, 2000 * Years_service, 0.25 * Salary}(EMPLOYEE)).$

Aggregate functions and grouping

- Common functions applied to collections of numeric values
- Include SUM, AVERAGE, MAXIMUM, and MINIMUM

Aggregate functions

- Group tuples by the value of some of their attributes
- Apply aggregate function independently to each group

$$\langle \text{grouping attributes} \rangle \mathcal{F} \langle \text{function list} \rangle (R)$$

Aggregate functions

Figure 6.10

The aggregate function operation.

- $\rho_{R(Dno, No_of_employees, Average_sal)}(Dno \int COUNT Ssn, AVERAGE Salary(EMPLOYEE)).$
- $Dno \int COUNT Ssn, AVERAGE Salary(EMPLOYEE).$
- $\int COUNT Ssn, AVERAGE Salary(EMPLOYEE).$

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125